

Factory Planning

Problem

- Find the most profitable production plan (maximize)

Sets

- Products: p
- Mashines: m
- Months: t

Parameters

- Profit: $Profit_p$
- Worktime requirements: $ProcessTime_{m,p}$
- Number of machines: $NumMach_m$
- When are the machines repaired:
 $MachineRepairs_{m,t}$
- The amount of processing time for product p on machine m: $ProcessTime_{m,p}$
- Demand, i.e. max sale of product p: $Demand_{p,t}$
- No. of hours of work pr. month:
 $WorkingHoursPrMonth = 2 \cdot 8 \cdot 24$
- Storage capacity: $StorageCapacity = 100$
- Storage cost pr. month pr. unit: $StorageCost = 0.5$
- Required end stock: $EndStock = 50$

Decision variables

- $x_{p,t}$: ammount of product p produced in month t
- $y_{p,t}$: ammount of product p sold in month t
- $v_{p,t}$: ammount of product p stored in month t

Model

Objective:

- The objective is to maximize profit:

$$\sum_p \sum_t Profit_p \cdot y_{p,t} - StorageCost \cdot v_{p,t}$$

Constraints:

- Machine capacity limit:

$$\sum_p ProcessTime_{m,p} \cdot x_{p,t} \leq (NumMach_m - MachineRepairs_{m,t}) \cdot WorkingHoursPrMonth \quad \forall m, t$$

- Balance constraint (notice we work with ULTIMO storage !):

$$v_{p,t} = v_{p,t-1} + x_{p,t} - y_{p,t} \quad \forall p, t$$

- Storage capacity limit:

$$v_{p,t} \leq StorageCapacity \quad \forall p, t$$

- End storage requirement:

$$v_{p,jun'} = EndStock \quad \forall i, t$$

- Limit on sale

$$y_{p,t} \leq Demand_{p,t} \quad \forall p, t$$

Notice that the costs depend on the storage strategy ! (We use ULTIMO storage here). In the following pages, we show first the Julia program for ULTIMO storage, then PRIMO storage.

The full model in Julia/JuMP with ULTIMO storage constraints , available with the name

FactoryPlanning_ULTIMO_ANS.jl
from the book web-site, is given below:

```

#####
# Factory Planning Assignment ULTIMO-STORAGE, "Mathematical Programming Modelling" (42112)

#####
# Intro definitions
using JuMP
using HiGHS
#####

#####
# PARAMETERS
Machines = ["Grinding","Vertical drilling",
            "Horizontal drilling","Boring","Planing"]
M=length(Machines)
Products = ["PROD1","PROD2","PROD3","PROD4",
            "PROD5","PROD6","PROD7"]
P=length(Products)
Time = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
T = length(Time)

# data
Profit = [10 6 8 4 11 9 3]
ProcessTime = # machine,product
[0.5 0.7 0.0 0.0 0.3 0.2 0.5; # Grinding
 0.1 0.2 0.0 0.3 0.0 0.6 0.0; # Vertical drilling
 0.2 0.0 0.8 0.0 0.0 0.0 0.6; # Horizontal drilling
 0.05 0.03 0.0 0.07 0.1 0.0 0.08; # Boring
 0.0 0.0 0.01 0.0 0.05 0.0 0.05 # Planing
]
MachineRepairs = # time,machine
#G  VD  HD  B  P
[1.0 0.0 0.0 0.0 0.0 ; # Jan
 0.0 0.0 2.0 0.0 0.0 ; # Feb
 0.0 0.0 0.0 1.0 0.0 ; # Mar
 0.0 1.0 0.0 0.0 0.0 ; # Apr
 1.0 1.0 0.0 0.0 0.0 ; # May
 0.0 0.0 1.0 0.0 1.0 # Jun
]

NumMach = [4 2 3 1 1]

```

```

StorageCap = 100
StorageCost = 0.5
EndStock = 50
WorkingHoursPrMonth = 24*8*2 # no working days, 2 shifts of 8 hours

Demand = # month,product
[500      600      300      200      0      500;
 1000     500     600     300     100     500;
 300      200      0      400     500     100;
 300      0      0      500     100     300;
 800      400     500     200     1000    1100;
 200      300     400      0      300     500;
 100      150     100     100      0      60;
]

#####

#####
# Model
factoryplanning = Model(HiGHS.Optimizer)

# production of product p in month t
@variable(factoryplanning, x[1:P,1:T] >= 0)

# sale of product p in month t
@variable(factoryplanning, y[1:P,1:T] >= 0)

# storage of product p in THE END of month t
@variable(factoryplanning, s[1:P,1:T] >= 0)

# maximize profit
@objective(factoryplanning, Max,
            sum(Profit[p]*y[p,t] for p=1:P, t=1:T) -
            sum(StorageCost*s[p,t] for p=1:P, t=1:T))

# machine limit constraint
@constraint(factoryplanning, [m=1:M,t=1:T],
            sum(ProcessTime[m,p]*x[p,t] for p=1:P) <=
            (NumMach[m] - MachineRepairs[t,m])*WorkingHoursPrMonth)

# storage balance constraint
@constraint(factoryplanning, [p=1:P, t=1:T],
            s[p,t] == x[p,t] - y[p,t] + (t>1 ? s[p,t-1] : 0) )

# end storage

```

```

@constraint(factoryplanning, [p=1:P],
            s[p,6] == EndStock)

# max storage
@constraint(factoryplanning, max_storage_con[p=1:P, t=1:T],
            s[p,t] <= StorageCap)

# demand limit
@constraint(factoryplanning, [p=1:P, t=1:T],
            y[p,t] <= Demand[p,t])

print(factoryplanning)

#####

# solve
optimize!(factoryplanning)
println("Termination status: $(termination_status(factoryplanning))")
#####

# Report results
println("-----");
if termination_status(factoryplanning) == MOI.OPTIMAL
    println("RESULTS:\n")
    println("objective = $(objective_value(factoryplanning))")
    println(dual.(max_storage_con))
else
    println(" No solution")
end
println("-----");
#####

println("Successfull end of $(PROGRAM_FILE)")
#####

```

Comments to the Julia/JuMP program:

- Notice: The smart way the planned machine maintenance is included: $NumMach_m - MachineRepairs_{m,t}$. This makes the FactoryPlanning2 assignment much easier.
- Notice: The reason that it is ULTIMO storage is because in the constraint:

$v_{p,t} = v_{p,t-1} + x_{p,t} - y_{p,t}$ this months storage has the same time index as x and y.

The full model in Julia/JuMP with PRIMO storage constraints , available with the name

FactoryPlanning_PRIMO_ANS.jl

from the book web-site, is given below:

```

*****
# Factory Planning Assignment PRIMO-STORAGE, "Mathematical Programming Modelling" (42112)
*****
# Intro definitions
using JuMP
using HiGHS
*****

*****
# PARAMETERS

Machines = ["Grinding","Vertical drilling",
            "Horizontal drilling","Boring","Planing"]
M=length(Machines)
Products = ["PROD1","PROD2","PROD3","PROD4",
            "PROD5","PROD6","PROD7"]
P=length(Products)
Time = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
T = length(Time)

# data
Profit = [10 6 8 4 11 9 3]
ProcessTime = # machine,product
    [0.5 0.7 0.0 0.0 0.3 0.2 0.5; # Grinding
     0.1 0.2 0.0 0.3 0.0 0.6 0.0; # Vertical drilling
     0.2 0.0 0.8 0.0 0.0 0.0 0.6; # Horizontal drilling
     0.05 0.03 0.0 0.07 0.1 0.0 0.08; # Boring
     0.0 0.0 0.01 0.0 0.05 0.0 0.05 # Planing
    ]
MachineRepairs = # time,machine
    #G  VD  HD  B  P
    [1.0 0.0 0.0 0.0 0.0 ; # Jan
     0.0 0.0 2.0 0.0 0.0 ; # Feb
     0.0 0.0 0.0 1.0 0.0 ; # Mar

```

```

0.0  1.0  0.0  0.0  0.0  ; # Apr
1.0  1.0  0.0  0.0  0.0  ; # May
0.0  0.0  1.0  0.0  1.0  # Jun
]

NumMach = [4 2 3 1 1]

StorageCap = 100
StorageCost = 0.5
EndStock = 50
WorkingHoursPrMonth = 24*8*2 # no working days, 2 shifts of 8 hours

Demand = # month,product
[500      600      300      200      0      500;
 1000      500      600      300      100      500;
 300      200      0      400      500      100;
 300      0      0      500      100      300;
 800      400      500      200      1000      1100;
 200      300      400      0      300      500;
 100      150      100      100      0      60;
]

#####

#####
# Model
factoryplanning = Model(HiGHS.Optimizer)

# production of product p in month t
@variable(factoryplanning, x[1:P,1:T] >= 0)

# sale of product p in month t
@variable(factoryplanning, y[1:P,1:T] >= 0)

# storage of product p in THE END of month t
@variable(factoryplanning, s[1:P,1:T] >= 0)

# maximize profit
@objective(factoryplanning, Max,
            sum(Profit[p]*y[p,t] for p=1:P, t=1:T) -
            sum(StorageCost*s[p,t] for p=1:P, t=1:T))

# machine limit constraint
@constraint(factoryplanning, machine_limit_con[m=1:M,t=1:T],
            sum(ProcessingTime[m,p]*x[p,t] for p=1:P) <=

```

```

        (NumMach[m] - MachineRepairs[t,m])*WorkingHoursPrMonth)

# storage balance constraint
@constraint(factoryplanning, balance_con[p=1:P, t=1:T-1],
    s[p,t] == (t>1 ? x[p,t-1] - y[p,t-1] + s[p,t-1] : 0) )

# end storage
@constraint(factoryplanning, end_storage_con[p=1:P],
    s[p,6] + x[p,6] - y[p,6] == EndStock)

# max storage
@constraint(factoryplanning, max_storage_con[p=1:P, t=1:T],
    s[p,t] <= StorageCap)

# Demand limit
@constraint(factoryplanning, Demand_lim_con[p=1:P, t=1:T],
    y[p,t] <= Demand[p,t])

print(factoryplanning)
#####

#####
# solve
optimize!(factoryplanning)
println("Termination status: $(termination_status(factoryplanning))")
#####

#####
# Report results
println("-----");
if termination_status(factoryplanning) == MOI.OPTIMAL
    println("RESULTS:\n")
    println("objective = $(objective_value(factoryplanning))")
else
    println(" No solution")
end
println("-----");
#####

#####
println("Successful end of $(PROGRAM_FILE)")
#####

```